FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

IEC 61499 Compliant Web-based IDE for Function Block Design

Pedro da Cunha Teixeira e Faro Beirão

WORKING VERSION

Mestrado em Engenharia Informática e Computação

Supervisor: Prof. Rui Pinto

June 5, 2026

"Programming is not a zero-sum game. Teaching something to a fellow programmer doesn't take it away from you. I'm happy to share what I can, because I'm in it for the love of programming."

John Carmack

Contents

1 Introduction **1**
- 1.1 Context . . . 1
- 1.2 Motivation . . . 2
- 1.3 Problem . . . 3
- 1.4 Research Questions . . . 3

2 Literature Review **4**
- 2.1 Methodology . . . 4
  - 2.1.1 Search and Selection . . . 4
- 2.2 Related Work . . . 5

3 State of the Art **7**
- 3.1 Ecosystem Summary . . . 7
  - 3.1.1 Open-Source . . . 7
  - 3.1.2 Commercial . . . 8
- 3.2 Local-first development environments . . . 8
- 3.3 Evolution of Web Based Tools . . . 8
- 3.4 Gap Analysis . . . 9

4 Proposed Solution **11**
- 4.1 Approach . . . 11
- 4.2 Methods . . . 12
  - 4.2.1 Research Design . . . 12
  - 4.2.2 Evaluation . . . 12
- 4.3 Workplan . . . 12

5 Prototype Development **14**
- 5.1 Runtime Environment . . . 14
- 5.2 Server . . . 15
- 5.3 Frontend . . . 16
  - 5.3.1 Devices Configuration View . . . 17
  - 5.3.2 Function Blocks Editor View . . . 18
  - 5.3.3 Graph View . . . 21
  - 5.3.4 Console View . . . 23
- 5.4 Real-Time Collaboration . . . 23
- 5.5 Deployment . . . 25
  - 5.5.1 Synchronise with DINASOREs . . . 25
  - 5.5.2 Send Commands . . . 26

5.5.3 Console . . . 29
5.5.4 Watch . . . 29
5.6 Distribution . . . 31

6 Validation **32**
6.1 4diac IDE User Experience . . . 32
6.1.1 Questionnaire Design . . . 32
6.1.2 Results . . . 33
6.1.3 Discussion . . . 34
6.2 Test Program Design . . . 34
6.2.1 Available resources . . . 34
6.2.2 Event and Data Flow . . . 34
6.2.3 Expected Output . . . 35
6.3 Objective Validation . . . 35
6.3.1 Functional Correctness . . . 35
6.3.2 Performance . . . 36
6.4 Subjective Validation . . . 38

7 Discussion **39**

References **40**

List of Figures

1.1 *IEC 61499* function block interface [6] 2
1.2 4diac IDE [1] 2

2.1 Example query on *ScienceDirect* with 136 results [16] 5

4.1 Gantt chart timeline for the proposed work 13

5.1 *DINASORE*'s architecture [12]. 15
5.2 Communication through the server 16
5.3 Devices Configuration View 18
5.4 Example function block defined via *xml* 19
5.5 Example function block defined via *xml* 20
5.6 Function Blocks Editor View 21
5.7 *DINASORE* communication format 26
5.8 Watching an output slot in the *Graph View* 31

6.1 Test design represented in *4diac IDE* 34

List of Tables

5.1 IEC 61131-3 Data Types [5] 22

List of Listings

5.1 Example *xml* definition of a function block 18
5.2 Example *Python* code of a function block 19
5.3 Structure of the Nodes serialized map 24
5.4 Structure of the Links serialized map 24
5.5 Sync configuration 25
5.6 build_message() in Python 26
5.7 build_message() in JavaScript 27
5.8 Setup deployment commands 27
5.9 `CREATE` nodes deployment commands 28
5.10 `CREATE` links deployment commands 28
5.11 `START` deployment command 28
5.12 `Response` after a deployment command is sent 29
5.13 `CREATE` watches after deploying 29
5.14 Request to `READ` watches 30
5.15 Response to `READ` watches 30

List of Acronyms

CI	Continuous Integration
CPPS	Cyber-Physical Production Systems
CRDT	Conflict-free Replicated Data Type
FBD	Function Block Diagram
FEUP	Faculdade de Engenharia da Universidade to Porto
ICS	Industrial Control Systems
IDE	Integrated Development Environment
MEEC	Master in Electrical and Computer Engineering
PLC	Programmable Logic Controller
RTE	Runtime Environment
SINF	Information Systems
SOA	Service-Oriented Architectures
UI	User Interface

Chapter 1

Introduction

1.1 Context

Cyber-Physical Production Systems (CPPS) [10] are complex systems that integrate physical processes with digital technologies to optimise production processes. Several software engineering approaches and technologies are utilised in the development and operation of CPPS. One of such approaches is Function Block Diagram (FBD) [3], which are graphical programming languages used in industrial automation and control systems. FBD uses graphical function blocks to represent the logic and control of a system. The function blocks are connected in a network to form a control program. One of the benefits of FBD is that it provides a visual representation of the control logic, which can be easier to understand and debug than traditional textual programming languages. FBD is also modular, which means that function blocks can be reused in multiple programs, reducing development time and improving code maintainability. Also, FBD is often used in conjunction with Programmable Logic Controllers (PLCs) to control machinery and processes in manufacturing and other industrial applications [11, 13]. FBD programs are not as simple to create and edit as traditional text-based languages since they require the use of specialised software tools that allow for the manipulation of function blocks [22].

In industrial automation, it is common to use the *IEC 61499* standard, which defines an event-driven FBD approach for designing control software [17]. Unlike traditional PLC programming languages, *IEC 61499* emphasises modularity, reusability, and explicit separation of events and data, allowing developers to design complex distributed systems in a scalable and maintainable way. Function blocks define both algorithms and interfaces for inputs, outputs, and events, making them ideal for CPPS development [4]. This approach is particularly suitable for distributed automation systems, where control logic must respond to events generated by multiple devices or sensors in real time. An example of such a function block interface is shown in Figure 1.1, highlighting the event inputs, outputs, and data connections that enable flexible interaction between system components.

While Runtime Environments (RTEs) enable the execution of function blocks compliant with the standard, Integrated Development Environments (IDEs) are used to design, create, and deploy

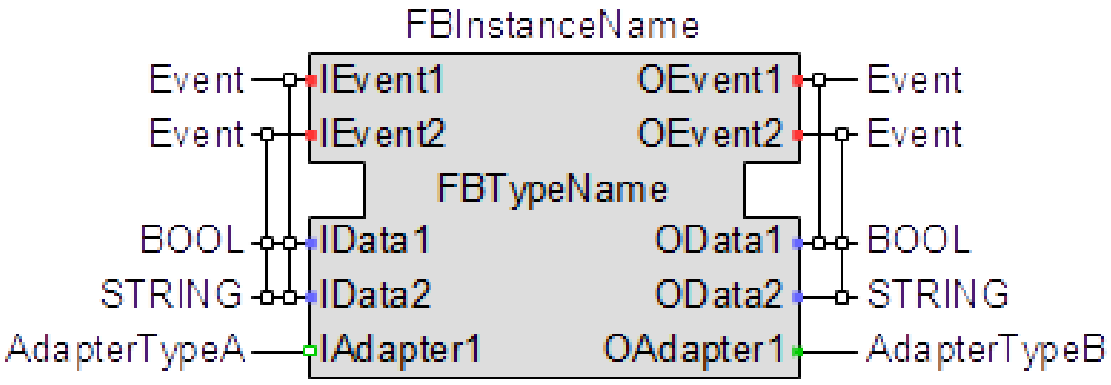


Figure 1.1: *IEC 61499* function block interface [6]

function block pipelines to CPPS. These IDEs offer an User Interface (UI) with features and tools that facilitate the development, testing, and deployment of *IEC 61499*-compliant control systems. They are graphical editors, like shown in Figure 1.2, for creating function block diagrams with simulation capabilities and additional functionalities to streamline the development process.

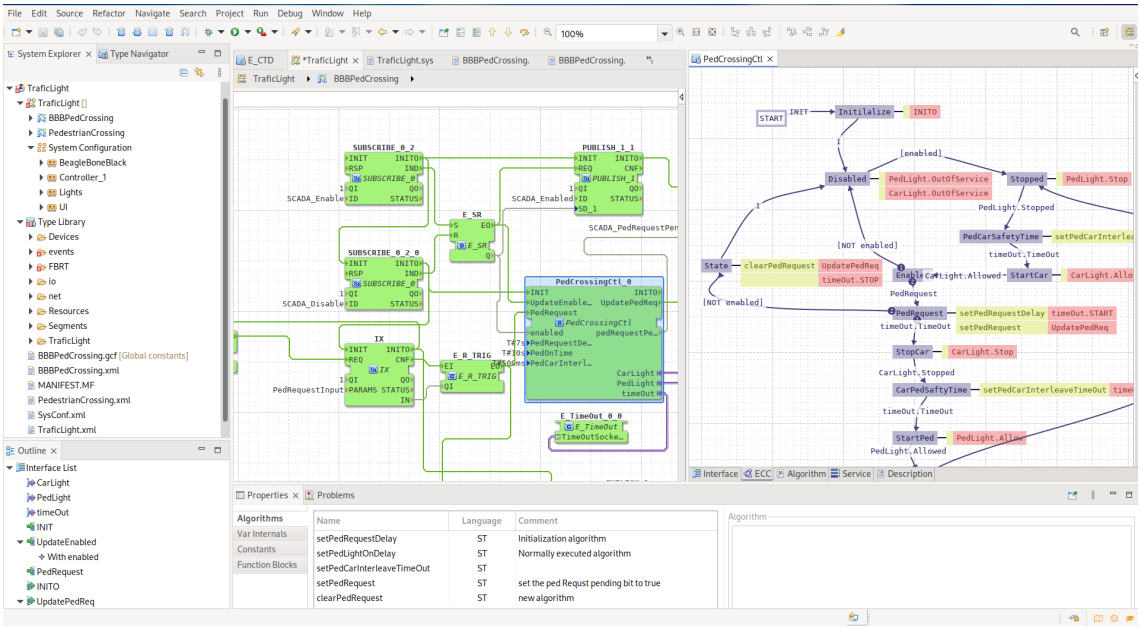


Figure 1.2: 4diac IDE [1]

1.2 Motivation

The evolution of CPPS has increased the demand for flexible and intelligent control solutions capable of integrating physical processes with digital technologies. FBD and the *IEC 61499* standard offer a structured, modular approach to building such distributed automation systems. To fully leverage these benefits, development tools must evolve alongside industrial needs, supporting more accessible, collaborative, and scalable workflows.

1.3 Problem

Existing *IEC 61499* development environments lack modern capabilities such as remote access, real-time collaboration, and seamless integration with cloud infrastructures. These limitations hinder efficient development, maintenance, and deployment of control applications within distributed CPPS. As a result, current tools fall short in supporting the flexibility and scalability required by certain modern industrial automation systems. Besides this, the complexity of these current IDEs rules them out for many simpler use cases where the time wasted on setting up and learning them is not justifiable.

1.4 Research Questions

This research seeks to investigate how modern web and distribution technologies can be leveraged to enhance the development, deployment, and operation of *IEC 61499*-compliant systems. This study is guided by the following main research question:

> How can we design IDEs that improve the development, deployment, and collaboration processes for *IEC 61499* applications in distributed CPPS?

To further refine the scope, the following sub-question is proposed:

> What architectural and technological requirements must be considered when designing an IDE for *IEC 61499*?

Chapter 2

Literature Review

This chapter reviews prior work related to *IEC 61499*-based development and the used IDEs. The goal is to better understand the landscape, which will then allow the critical review of the state of the art.

2.1 Methodology

A targeted literature review was carried out, focusing on studies addressing *IEC 61499* development tools, function blocks, CPPS, and development environments, to then critically reflect upon their findings to help the writing of this dissertation and the planning of the final product.

2.1.1 Search and Selection

The search was conducted in digital libraries *ScienceDirect*, *IEEE Xplore*, *ACM Digital Library*, among others, as well as research paper aggregators like *Semantic Scholar*, *Scopus* and *Web of Science* using combinations of the following terms:

- *IEC 61499*
- FBD
- IDE
- CPPS

Following the search, a manual review process was conducted to refine and validate the results. Titles and abstracts were first screened to understand the relevance of the articles with respect to this dissertation. The publications were then examined in greater detail through full-text review, the reference lists of selected papers were manually inspected to identify relevant works that may not have been captured by the initial keyword search.

Articles mentioning new IDE proposals [17] were immediately included as their novel ideas were of great interest for a gap analysis on this ecosystem. These papers usually already contained

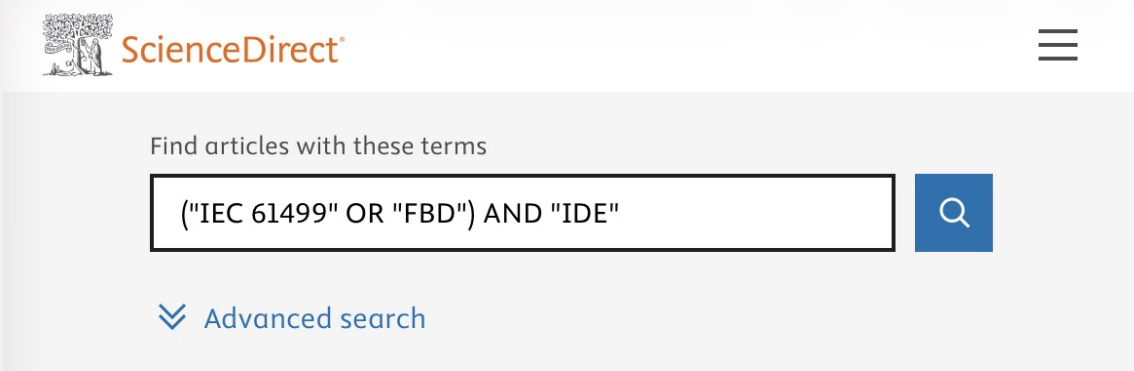


Figure 2.1: Example query on *ScienceDirect* with 136 results [16]

a gap analysis of their own, which allowed for a literature review that was consistent with the prevalent opinion of the scientific community.

Furthermore, terms related to the state of the art that were discovered mid search were also included in some queries. The objectives of doing this were to get a better personal understanding of the scene, as well as to find interesting references that could be included in this dissertation to backup the written claims. Some of the terms used were:

- 4diac
- collaboration
- real-time
- remote

2.2 Related Work

It should be interesting to mention some papers that were utilised to get the knowledge used for writing this dissertation and which will be referenced throughout this whole document.

Monostori [10] provides a comprehensive analysis of CPPS, examining the theoretical foundations and practical implications of tightly coupling computational intelligence with physical manufacturing processes. The author explores how cyber-physical integration enables real-time monitoring, adaptive control, and autonomous decision-making in production systems, framing CPPS as a cornerstone of the fourth industrial revolution. This work establishes the concepts wording and architectural principles that help understand much of the research in this area, and it serves as a reference for understanding why the flexible distributed control design of *IEC 61499* is necessary to realize the vision of CPPS.

Building upon this foundation, Torres, Gonçalves, and Pinto [18] conducted a literature review examining how *IEC 61499* can be leveraged specifically for CPPS development, mentioning possible enhancements with the use of a cloud-based Service-Oriented Architectures (SOA) approach. Their paper examines the current state of adoption of the standard within industrial automation

research, identifying recurring architectural patterns in *IEC 61499* based CPPS implementations and highlights challenges such as real-time performance, modularity and privacy.

Zoitl, Strasser, and Ebenhofer [22] demonstrated the use of the open source *4DIAC* environment to develop modular and reusable *IEC 61499* applications, highlighting the benefits of flexibility and adaptability in industrial settings. Their work presents concrete design patterns for structuring function block networks, discusses the behaviour of *4diac FORTE* as the RTE and validates the approach through case studies in distributed control. The paper is interesting as it provides a practical perspective on the strengths and limitations of *4DIAC* as a development environment, insights that are directly relevant to the evaluation conducted in this dissertation.

Similarly, Pinto et al. [13] presented an industrial case study showing that function block oriented approaches improve system modularity and reduce development effort compared to traditional procedural programming. Their study documents the migration of a production cell from a sequential control architecture to an *IEC 61499* function block model using the *DINASORE* RTE, using integration time as the main validation metric and demonstrating improved reconfigurability when production requirements changed. The results show that the encapsulation of behaviour and communication interfaces within function blocks leads to more maintainable and extensible software.

Some more recent studies explore remote web based execution and editing of FBD. Rohat and Popescu [14] proposed an early approach to remote monitoring and interaction with *IEC 61499* applications through a web interface, identifying the practical benefits of browser access in distributed industrial settings. While the work predates many modern web technologies, it established the motivation for moving development and monitoring capabilities to the browser. More recently, Lyu et al. [9] revisited this direction by proposing fully remote cloud based platform for *IEC 61499* development, embedding an IDE in a frontend web page, removing the need for users to install anything in their workstations. This is possible by moving the processing into a centralised cloud platform.

Chapter 3

State of the Art

This chapter provides a comprehensive analysis of the current landscape of development environments for CPPS and the *IEC 61499* standard. It categorises and critically examines the existing development tools, while looking for possible gaps in the ecosystem.

3.1 Ecosystem Summary

Currently, the development ecosystem for *IEC 61499* is composed almost exclusively of desktop-based IDEs. These tools are typically monolithic applications installed locally on engineering workstations, requiring each member of a development team to independently install, configure, and maintain the software on their own machine. The tools themselves tend to be resource intensive, which creates demands for the hardware, making them less accessible to users working on modest, shared or virtual machines. There is also the issue of platform dependency, as a niche and focused field, some IDEs are not crossplatform, forcing users to use virtual machines to be able to use them, further intensifying performance issues.

3.1.1 Open-Source

The most prominent open source implementation is the *4diac IDE* (based on the *Eclipse* framework) which supports a variety of RTEs such as *4diac FORTE* and *DINASORE*. This IDE is very feature rich at the cost of increased complexity, resulting in a steep learning curve, which makes it more difficult to use [20]. The *Eclipse* foundation on which it is built is itself a heavyweight platform, and while this grants access to a broad plugin ecosystem, it also means that the IDE inherits the framework's architectural complexity. Navigating the interface to accomplish basic tasks such as creating a new function block type, designing the flow and deploying to a RTE can require substantial initial investment before productive work can begin.

An alternative is *FBME* which is a much simpler modelling environment [17]. Built as a plugin for the *JetBrains MPS* language workbench, while easier to use than *4diac* IDE, it still is a local-first development environment, the importance of this will be explained in Section 3.2.

3.1.2 Commercial

The commercial side is occupied by *EcoStruxure Automation Expert*, *ISaGRAF Workbench* and other development environments that usually have their own built-in proprietary RTE. This tight coupling between the engineering tool and the runtime gives commercial vendors significant control over the end-to-end development experience, which typically results in polished, production-ready tooling with strong vendor support. However, it also introduces considerable lock-in: projects developed in one commercial environment are not easily portable to a different runtime or IDE and the proprietary nature of the underlying RTE limits the ability to inspect, extend, or adapt behaviour.

Commercial tools are also constrained by their licensing models. Per seat licensing is common, meaning that the cost of the toolchain scales directly with the size of the team and can not be viable for smaller organisations, academic research groups, or early stage projects. This creates a tiered ecosystem in which well resourced industrial organisations have access to mature tooling, while smaller teams are restricted to the open source alternatives, each of which carries its own set of limitations as described above.

3.2 Local-first development environments

All of the tools described in the previous section share a fundamental design decision: they play around a local-first model, in which the project state lives on a single workstation and the tool itself is a desktop application that must be installed and executed locally. While this model has historically been the norm in industrial automation tooling, it introduces a set of limitations that become problematic as CPPS projects grow in scale and involve more distributed engineering teams.

The most immediate consequence is the absence of real-time collaboration. When two engineers need to work on the same *IEC 61499* application simultaneously, they must coordinate manually, typically by dividing the project into separate files or modules and merging their changes afterwards. Unlike regular text based code, FBD programs are stored in structured data files, *XML* usually, which makes version control systems like *Git* not appropriate for collaboration in this standard [7].

Local-first tools also require the IDE to maintain awareness of the deployment target's state. Without a shared backend, each engineer must independently configure their local environment to point to the correct runtime endpoints and therefore there is no single source of truth for the current deployment state of the system.

3.3 Evolution of Web Based Tools

The software engineering domain has seen a massive shift from desktop IDEs to cloud based environments. This trend is now influencing the domain of Industrial Control Systems (ICS), and

while recent studies have proposed a SOA to integrate *IEC 61499* with cloud layers [18], the engineering tools themselves remain still desktop bound.

In general software development, tools like *Visual Studio Code* have started offering web based counterparts to their desktop applications [19] in search of a more flexible user experience. Running entirely within the browser, these environments allow engineers to open, edit, and navigate codebases from any device without installation. They integrate naturally with cloud hosted codebases and Continuous Integration (CI) pipelines. In fact, newer development environments like *Zed* make it a key goal of their products to have web platform support [21], treating browser delivery not as an afterthought but as a primary design constraint.

Web based editors are also often easier to extend with real time collaboration because they operate within a network architecture, leverage standardised browser communication technologies such as WebSockets, provide uniform deployment through URLs, and naturally integrate with centralised synchronisation services. These characteristics reduce many of the distribution and interoperability challenges that desktop applications must address separately [15].

The move towards web based tooling is not limited to software development environments. Across a wide range of professional domains, browser-native applications have gradually replaced traditional desktop software. Collaborative document editors such as *Google Docs* allow multiple users to edit the same document simultaneously, with changes synchronised in real time. Similarly, design and prototyping platforms such as *Figma* have demonstrated that even complex graphical workflows can be delivered entirely through the browser while supporting collaboration and centralised asset management. Drawing tools like *Excalidraw* and *draw.io* further illustrate how browser based applications can provide sophisticated modelling and visualisation capabilities without requiring local installation, while providing live collaboration features that their desktop predecessors did not have.

A common characteristic of these platforms is that they treat collaboration as an architectural design rather than an addon. By maintaining a shared representation of the artifact on a central service, users can concurrently make changes with minimal configuration. This contrasts with many traditional desktop applications, where collaboration often relies on exchanging files with separate version control and synchronisation mechanisms. As a result, web based tools have created a new expectation on collaboration.

3.4 Gap Analysis

Current *IEC 61499* tools face significant limitations that hinder modern CPPS development. These gaps paint a picture of an ecosystem that has matured in terms of standards compliance, but has lagged considerably in adapting to the workflows and expectations of contemporary engineering teams. The following points describe the most significant gaps:

- **Platform Dependency:** Existing solutions are monolithic desktop applications, lacking the accessibility and portability that we have come to expect. Engineers are required to install

and maintain native software on every machine from which they intend to work, creating friction at the start of projects and compounding over time as the team grows or its members change.

- **Absence of Real-Time Collaboration:** None of the existing tools provide support for real-time editing. Teams must rely on external version control systems and manual coordination to manage shared development, introducing overhead and increasing the risk of integration errors that would be avoided by a collaboration aware environment.
- **Lack of a Centralised System State:** Since tools are local-first, there is no single representation of a system's current state. This makes it difficult to maintain a consistent view of the system across a distributed team to integrate the development environment into automated deployment and monitoring pipelines.
- **Limited Accessibility:** Engineers cannot access a project without access to a fully configured workstation. This is a practical limitation in industrial settings, where on-site diagnosis of a running application may be necessary from a device in which no local tooling has been installed and the project is not set up.
- **Web Trends:** Despite a clear industry shift toward web based development tooling, there is currently no mature web solution for *IEC 61499*, leaving ICS engineering behind. The contrast with general software development, where fully featured browser IDEs are now common, represents both a gap and an opportunity. The techniques and infrastructure required to build such a tool are well understood, but they have not yet been applied to this domain.

Taken together, these gaps suggest that the ecosystem is well positioned to benefit from a lightweight, browser-accessible development environment that prioritises collaboration, low setup overhead, and integration with modern deployment workflows — without sacrificing the domain-specific features that *IEC 61499* engineers depend on.

Chapter 4

Proposed Solution

To address the limitations identified in existing *IEC 61499* development environments, this dissertation proposes the design of a web-based IDE for modelling *IEC 61499*-compliant applications. The goal is to bridge the gap between modern collaborative software engineering practices and industrial automation tooling used in CPPS.

Existing tools are predominantly desktop based and poorly suited for distributed development. In contrast, the proposed solution adopts a centralised web-based architecture that enables platform independence, simplified setup, simplified deployment, and real-time collaboration.

The main reason to propose using web technologies is to solve the issues that affect any local-first development environment. By having the IDE be hosted by a centralised server, the following is achieved:

4.1 Approach

The approach is derived from functional and non-functional requirements extracted from the *IEC 61499* standard, an analysis of existing tools and the gaps identified in the state of the art. Functional requirements include:

- Creation of function block types
- Design of the event and data flow
- Deployment to compliant RTEs
- Real-time collaboration

Non-functional requirements should ensure that the solution does not reflect negatively when compared with *4diac IDE* in some key aspects as well as bridge the gaps identified. These include:

- Platform independence
- Reduced setup complexity

- Improved usability for distributed teams
- Ease of extensibility for future improvements
- The webpage should load in less than 2 seconds
- Deployment should take just as long
- Should be able to handle 100 users concurrently

The effectiveness of the approach is assessed through the implementation of a functional prototype and its direct comparison with current established IDEs, focusing on usability, accessibility and workflow efficiency. This evaluation demonstrates the feasibility of applying web based principles to *IEC 61499* development and their potential benefits for distributed CPPS.

4.2 Methods

This section describes the research methodology adopted to address the research questions defined in this dissertation. Given the practical and design-oriented nature of the problem, the methodology focuses on the design, implementation, and evaluation of a piece of software. The chosen methods are appropriate for investigating how web-based technologies can enhance *IEC 61499* development environments, as they allow both the realisation of a concrete solution and the systematic assessment of its capabilities relative to existing tools.

4.2.1 Research Design

This research follows a qualitative, design-oriented research approach. The core of the research consists of the design and implementation of a functional prototype of the solution. Rather than aiming for statistical generalisation, the study focuses on demonstrating feasibility and identifying qualitative improvements over existing desktop-based environments. The prototype serves as a research artifact used to explore architectural decisions, usability considerations, and workflow implications.

4.2.2 Evaluation

Evaluation is conducted through qualitative analysis and comparative assessment against established *IEC 61499* IDEs. Criteria such as accessibility, usability, feature coverage, and support for collaborative workflows are used to assess how effectively the proposed solution addresses the limitations identified in the literature and state of the art.

4.3 Workplan

Figure 4.1 outlines the planned work for this dissertation. The schedule is organised into three main phases: Experimentation, Code Development, and Deliveries. During the Experimentation

phase, there will be a focus on studying the standard and running experiments to establish a solid technical foundation. The Code Development phase overlaps partially with Experimentation, ensuring that the software evolves in parallel with the experimental insights. Finally, the Deliveries phase spans the entire project duration and includes writing the dissertation and preparing the final presentation, ensuring that all outputs are documented and communicated effectively.

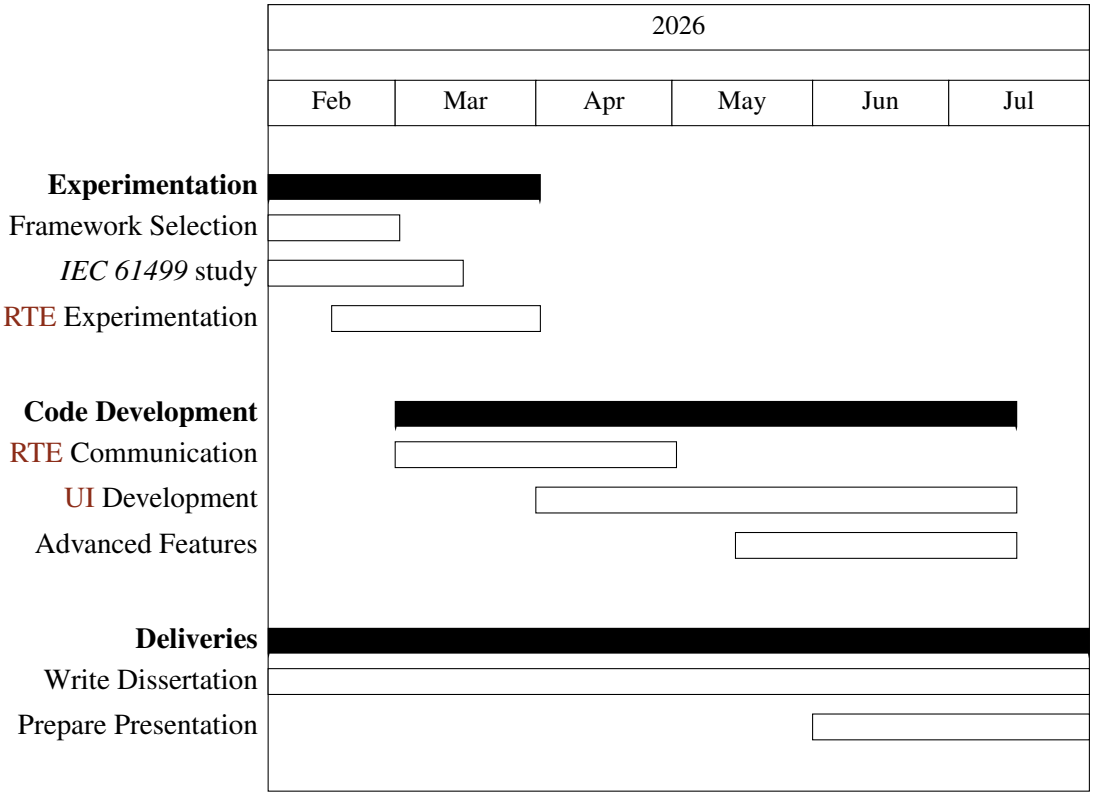


Figure 4.1: Gantt chart timeline for the proposed work

Chapter 5

Prototype Development

To be able to prove the proposed solution is viable and will successfully address the gap identified, a prototype needs to be developed and afterwards validated.

5.1 Runtime Environment

Out of all RTEs compatible with *IEC 61499*, this prototype will focus on supporting *DINASORE* (Dynamic INtelligent Architecture for Software and MOdular REconfiguration) [12]. It is much less complex than others, making it ideal for a proof of concept IDE. As an open-source project, *DINASORE* can be freely inspected and studied, which is particularly valuable in a research and prototyping context where flexibility is prioritised over real-time performance guarantees. This ease of study rules out the use of commercial RTEs like *EcoStruxure Automation Expert's "dPAC"* and *ISaGRAF Runtime*. Other open-source RTEs were considered, but none was as well-suited as *DINASORE*. *4diac FORTE* for example was way too extensive and complicated to be able to easily study its source code.

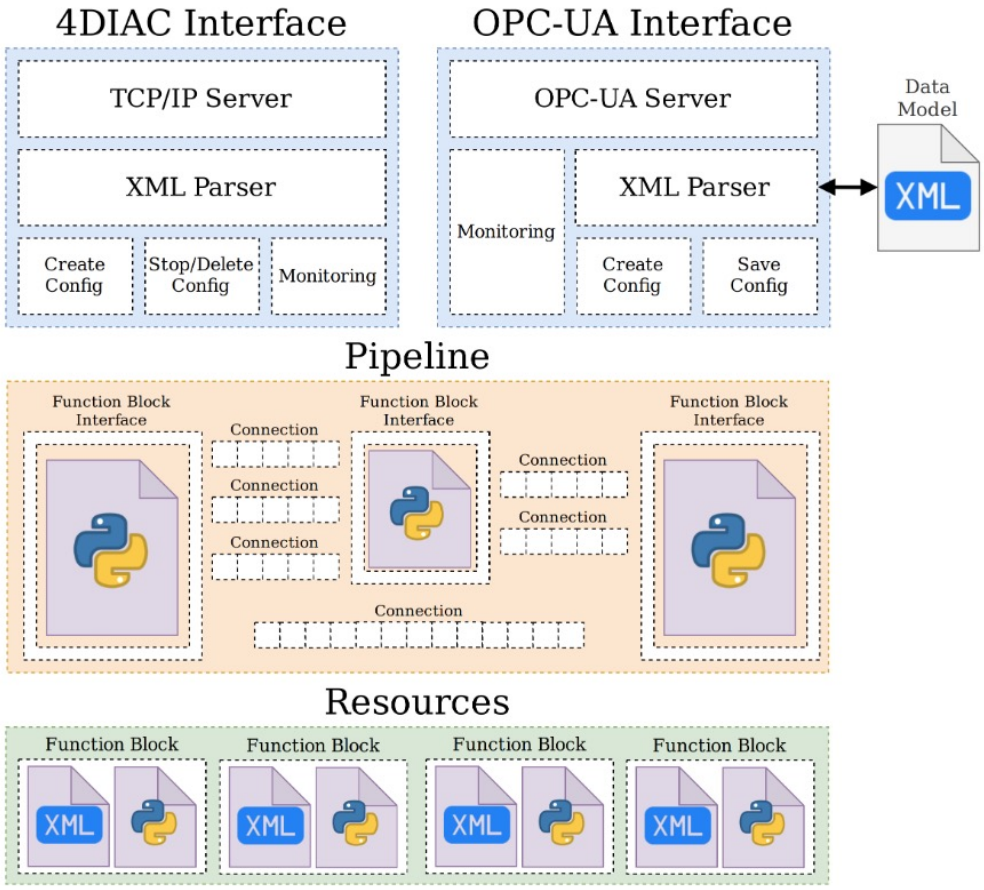


Figure 5.1: *DINASORE*'s architecture [12].

By default, *DINASORE* uses the port *61499* to receive commands from the IDE and send responses. This is important to note as it is the port that will be used as example throughout this document. The exact details of the communication implementation will be explained further in Section 5.5.

5.2 Server

The design begins by setting up a server that will serve the IDE interface as a webpage, store and manage all the data and handle communication with the RTEs. Taking into account the features we plan on implementing, *JavaScript* running on *Node.js* were chosen due to the extensive library ecosystem. For real-time collaboration we will need a library to run in both the server and the frontend, since *JavaScript* runs natively in the browser, it further reinforces that it should also be used in the server. The library choice will be explained in depth in Section 5.4, proving the previous train of thought.

Being the middleware, there are 4 different types of network communications that the server will need to handle, each with it's own requirements:

- **Serve webpage**: Communicates via *GET* requests to deliver the *html*, *css* and *js* files that make up the IDE interface.
- **Real-Time Collaboration**: Constant atomic updates via a library.
- **Requested deployment**: A team member sends a *POST* request, that tells the server to deploy.
- **Deployment sockets**: Communicates via sockets with each RTE to send commands and receive the corresponding responses.

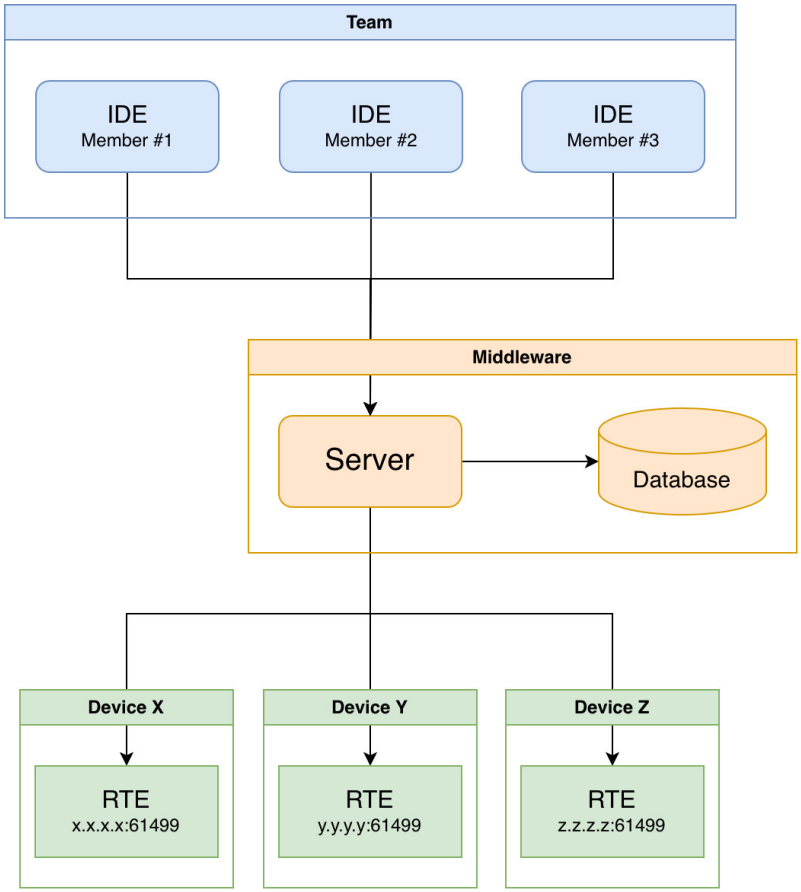


Figure 5.2: Communication through the server

5.3 Frontend

The frontend is the UI that will be used to create and modify the FBD programs.

Each subsection will first explain and describe what an IDE should implement and follow by showing how the prototype solved the proposed requirements.

5.3.1 Devices Configuration View

In the interest of distributed CPPS, *IEC 61499* allows function blocks to be mapped to different *DINASOREs*, so we need a clean way to define the resources that are available. Both the *DINASORE* port and the *SSH* port need to be defined for each defined since deployment communication will be made through both.

Each device should have several fields that the team can configure:

- **Name**: Human friendly name to quickly identify what device this is
- **Address**: The address of the device
- **DINASORE port**: The port where *DINASORE* is running
- **Color**: The color associated with this *DINASORE* instance, providing a visual distinction in the *Graph View* between function blocks mapped to different instances.
- **SSH port**: The open *SSH* port of the device.
- **SSH user** and **SSH password**: The *SSH* login credentials.
- **SSH path to DINASORE**: Path in the *SSH* filesystem pointing to where *DINASORE* is located.

With the existence of the middleware, this solution removes the requirement for each team member to configure the available *DINASORE* instances in their own IDE, and instead keeps a shared configuration in the server database.

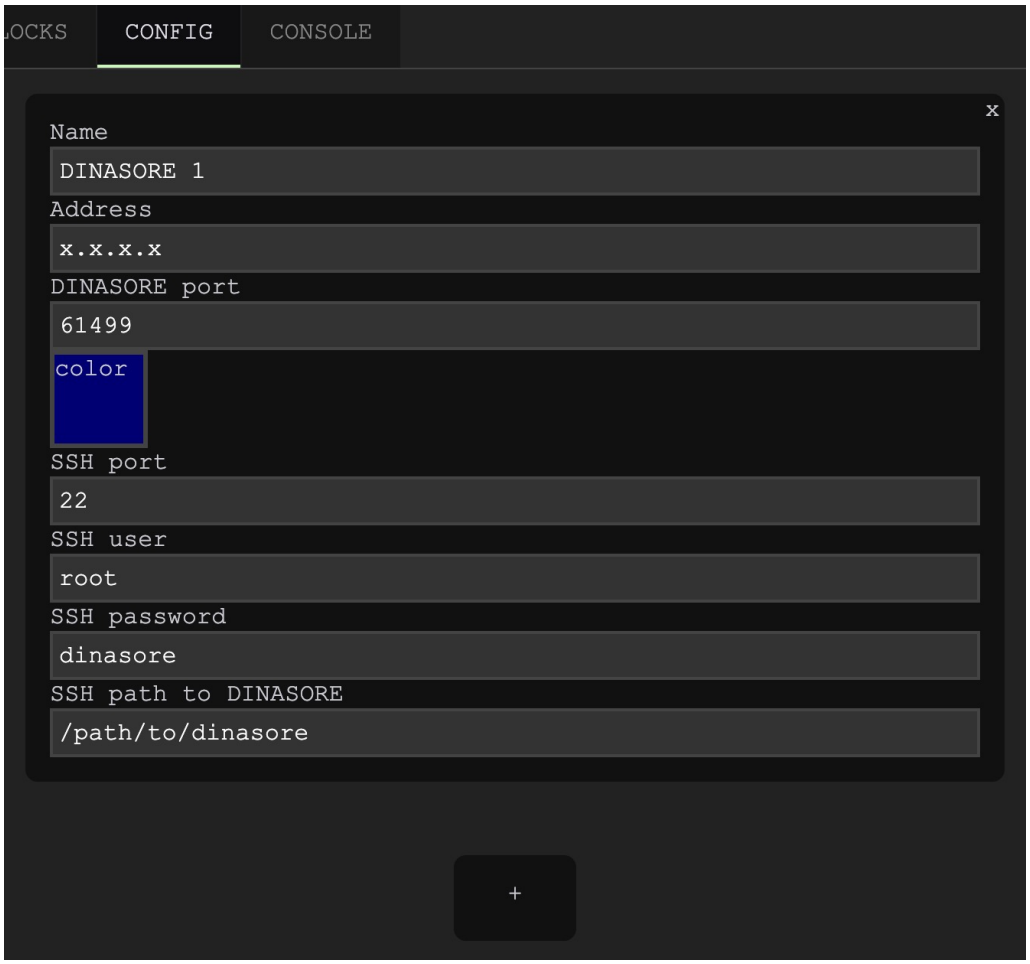


Figure 5.3: Devices Configuration View

5.3.2 Function Blocks Editor View

This is where individual function blocks are defined. It allows the team to specify the interface of each function block, its input and output data ports and events, as well as program the internal behaviour. This separation of interface and implementation follows the *IEC 61499* model of encapsulation, where a function block exposes a well-defined interface while keeping its internal logic self-contained.

Like in the other RTEs, in *DINASORE* the interface of the function block is defined in a *xml* format file with a *.fbt* extension, as seen in Listing 5.1 which results in Figure 5.4.

```
1 <FBType Name="NEW_FB_1">
2   <InterfaceList>
3     <EventInputs>
4       <Event Name="INIT" Type="Event"/>
5       <Event Name="READ" Type="Event">
6         <With Var="VARIABLE_1"/>
7         <With var="VARIABLE_2"/>
8         <With var="VARIABLE_3"/>
```

```
9       </Event>
10    </EventInputs>
11    <EventOutputs>
12      <Event Name="INIT_O" Type="Event"/>
13      <Event Name="READ_O" Type="Event">
14        <With Var="DATA_1"/>
15      </Event>
16    </EventOutputs>
17    <InputVars>
18      <VarDeclaration Name="VARIABLE_1" Type="STRING"/>
19      <VarDeclaration Name="VARIABLE_2" Type="REAL"/>
20      <VarDeclaration Name="VARIABLE_3" Type="INT"/>
21    </InputVars>
22    <OutputVars>
23      <VarDeclaration Name="DATA_1" Type="STRING"/>
24    </OutputVars>
25  </InterfaceList>
26 </FBType>
```

Listing 5.1: Example *xml* definition of a function block

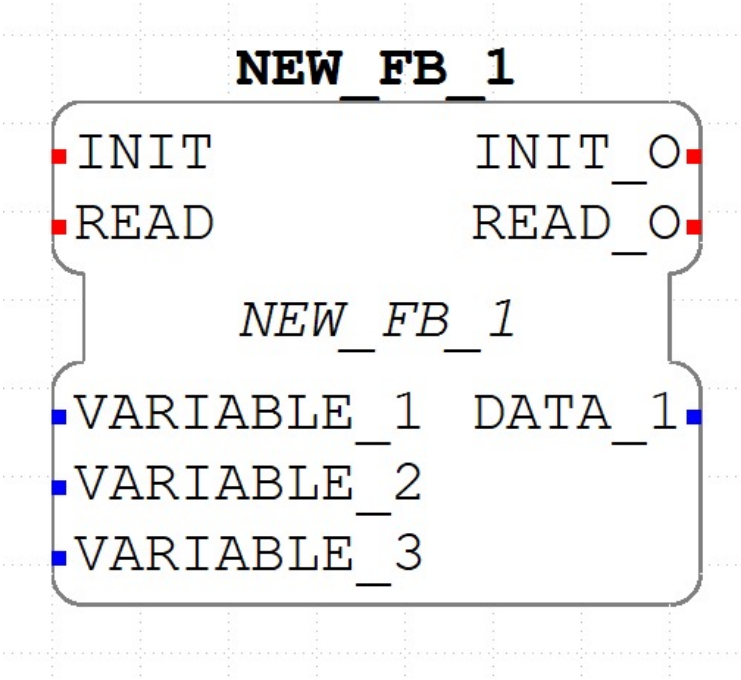


Figure 5.4: Example function block defined via *xml*

The internal behaviour of the function blocks in *DINASORE* is programmed in *Python*.

```
1 class NEW_FB_1:
2   def __init__(self):
3     pass
4
5   def schedule(self, event_name, event_value, variable_1, variable_2, variable_3):
```

```
6     if event_name == "INIT":
7       return [event_value, None, ""]
8
9     elif event_name == "READ":
10      return [None, event_value, ""]
```

Listing 5.2: Example *Python* code of a function block

For the IDEs to be able to detect and use the function block, both the *.fbt* file and the *.py* file need to be placed in a specific folder, right next to each other, like shown in Figure 5.5. In this prototype this is handled automatically in the server database, removing the need to manually move files like it is needed in *4diac IDE*.

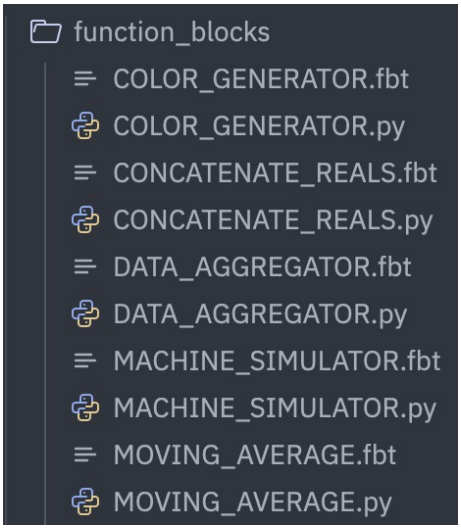


Figure 5.5: Example function block defined via *xml*

This view contains a split view text editor: a side for the *.fbt* file and the other for the *.py* file. It would be possible to create a visual editor for designing the interface, instead of having to write *xml* by hand, but the added complexity would not be justified at this prototyping stage. The *xml* format is sufficiently structured that a text editor provides an adequate experience, and a dedicated visual interface remains a natural direction for future development.

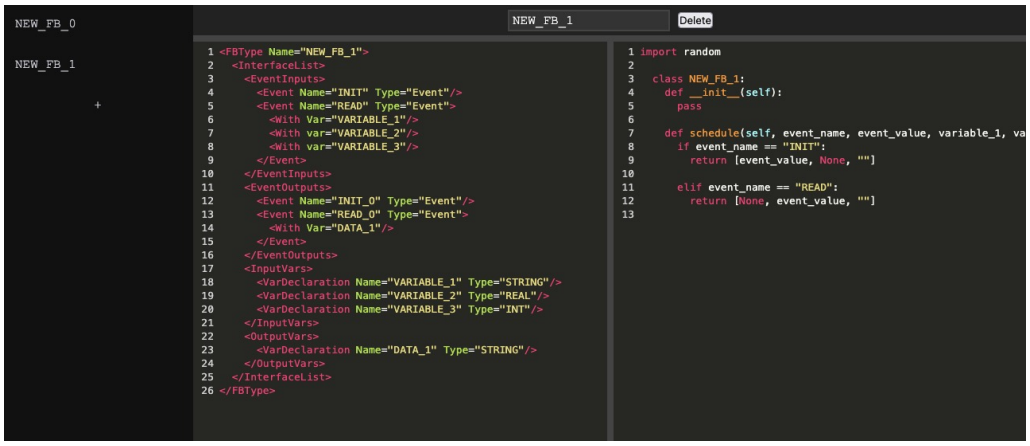


Figure 5.6: Function Blocks Editor View

5.3.3 Graph View

This view is the primary workspace for composing the application. Function blocks are placed on the canvas and connected together by drawing connections between their data and event ports, forming the application network. Event ports are commonly color coded red, while data ports are color coded blue.

In *DINASORE*, all *INIT* event ports are by default assigned to *EMB_RES*, the default resource that sends an event when the application is started. This removes the need to connect the *INIT* event ports, but the *READ* event port is not assigned by default and therefore needs to be connected in every function block. The `schedule()` function will only be called and generate output when an event is sent to the *READ* event port. The described event driven execution model is what allows an explicit specification of the execution order of function blocks. Periodic out of event execution of function blocks is also possible with the implementation of a looping function block that triggers events [6].

A function block will have some input and output data ports, each with its own associated data type. For example, an *INT* input port will not accept data from a *WORD* output port, and therefore should not be able to connect to it. It is also possible to set a default value for an input if no output is connected to it, there is an editable string field for this which gets converted to the expected data type by the RTE. The data types supported by *IEC 61499* are derived from and defined by *IEC 61131-3*.

Type	Description	Size
BOOL	Boolean	1-bit
SINT	Short integer	8-bit
INT	Integer	16-bit
DINT	Double integer	32-bit
LINT	Long integer	64-bit
REAL	Floating point	32-bit
LREAL	Long floating point	64-bit
STRING	ASCII string	Variable
WSTRING	Unicode string	Variable
TIME	Time interval	
DATE	Date	

Table 5.1: IEC 61131-3 Data Types [5]

Each function block can also be mapped to a *DINASORE* instance, with the color coding defined in the Devices Configuration View providing an immediate visual indication of how the application is distributed across the available devices.

This system was achieved in the prototype with the use of the *LiteGraph.js* library, a graph node editor for the browser. A lot of other libraries were considered, but this one was chosen due to the requirements of the proposed solution. Let us discuss the alternatives that were considered to understand why this choice is important:

- **Rete.js**: A very flexible and complete editor framework, but the added complexity makes it harder to integrate the desired features. It is also a more general purpose editor, meaning the multi-input and multi-output function blocks with specific data types are not as well supported.
- **xyflow**: A highly customizable framework composed of React Flow and Svelte Flow. While it provides excellent visual customisation and integration with frontend frameworks, it suffers from the same problems as *Rete.js*.
- **Drawflow**: A lightweight and simplistic library with a focus on data flow execution. Although simple to integrate, it lacks many features required for designing *IEC 61499* function blocks, such as robust type handling and multi-input and multi-output.
- **Baklava.js**: With a clean architecture and flexibility, this would be a good alternative, but since it specifically targets *Vue* and requires a more intricate setup, it was not the most appropriate choice for a prototype.
- **GoJS**: Very powerful commercial alternative. It offers advanced functionality and flexibility, but its licensing model make it less suitable for lightweight research prototypes and open development environments.

- **Node-RED**: A flow based programming environment. While conceptually similar, its architecture is more focused on runtime execution than on being a visual editor of structured data.

Among these alternatives, *LiteGraph.js* presented the best balance between simplicity, flexibility and ease of integration. Its builtin support for typed inputs and outputs, lightweight architecture and direct focus on visual data flow programming made it especially suitable for the requirements of the proposed solution. Even so, it was necessary to implement custom behaviour on top of some functions of *LiteGraph.js* to achieve the desired behaviour. The choice of the graph library is very important, since the requirements of *IEC 61499* differ significantly from those of most conventional graph editors, as such, special care must be taken to ensure that these requirements are properly supported.

5.3.4 Console View

This view will display the console logs that are generated when deploying. The contents of it will be discussed in detail in Section 5.5.

5.4 Real-Time Collaboration

One of the main goals of the proposed solution is to support collaborative development of *IEC 61499* applications. Since CPPS projects are often developed by multiple team members simultaneously, the code collaboration system must accommodate for this. Traditional version control systems such as *Git* are not well suited for collaborative editing of structured graph based data represented in formats such as *json* or *xml*. Although these formats are text based, small logical modifications in the application model can produce large text changes due to formatting changes, element reordering or nested structure updates. As a result, merge conflicts become frequent and difficult to resolve, especially when multiple team members edit the a graph at the same time. This limitation makes real time collaborative editing impractical, motivating the use of synchronisation approaches based on shared data structures and conflict free replication mechanisms such as Conflict-free Replicated Data Types (CRDTs).

In this prototype the collaborative editing system was implemented using *Yjs*, a CRDT framework designed for distributed collaborative applications. CRDT based systems allow users to concurrently edit shared data structures while automatically resolving conflicts in a deterministic manner.

The graph structure in *LiteGraph.js* representing the FBD application is synchronised through shared *Yjs* data structures. Operations such as creating function blocks, deleting connections, modifying inputs or moving nodes are propagated in real time to all connected clients. Each user therefore observes the same graph state with minimal delay. To integrate collaboration into this editor, the internal state changes of the graph were connected to the shared *Yjs* document. Whenever a local modification occurs in the graph, the corresponding operation is serialized and

applied to the shared collaborative state. Likewise, remote updates received from other users are reflected back into the local graph editor instance.

The serialized data for the graph structure can be divided into 2 maps (using the `Y.Map` structure), one for the nodes (shown in Listing 5.3) and one for the links (shown in Listing 5.4). In the nodes map, since the `title` of each node can be modified and the `type` can be the same across multiple nodes, we need to define a unique persistent id for each. Besides these, we need to also store the position of the node defined as `x` and `y` coordinates, the id of the *DINASORE* that this node is `mappedto` and the default `inputs` in case some data input slot is not being fed data from any link. As for the links map, each link must also have a unique persistent id, and then store the origin (represented by the `origin_id` node and the `origin_slot`) and the target (represented by the `target_id` node and the `target_slot`)

```
{
  'node_id_1': {
    type: 'SENSOR_SIMULATOR',
    title: 'SENSOR_SIMULATOR_1',
    x: 139, y: 526,
    mappedto: 'dinasore_id_1',
    inputs: { OFFSET: '' }
  },
  'node_id_2': {
    type: 'MOVING_AVERAGE',
    title: 'MOVING_AVERAGE_1',
    x: 487, y: 551,
    mappedto: 'dinasore_id_1',
    inputs: { WINDOW: '5', VALUE: '' }
  }
}
```

Listing 5.3: Structure of the Nodes serialized map

```
{
  'link_id_1': {
    origin_id: 'node_id_1',
    origin_slot: 'VALUE',
    target_id: 'node_id_2',
    target_slot: 'VALUE'
  },
  'link_id_2': {
    origin_id: 'node_id_1',
    origin_slot: 'READ_0',
    target_id: 'node_id_2',
    target_slot: 'RUN'
  }
}
```

Listing 5.4: Structure of the Links serialized map

5.5 Deployment

The deployment process is responsible for taking the application defined in the *Graph View* and bringing it to life across the configured *DINASORE* instances. It is broken down into a sequence of steps, each of which is described in the following subsections.

5.5.1 Synchronise with DINASOREs

The first step is to ensure that each *DINASORE* instance has access to the function blocks that it will need to run the application. This is done by sending all the *.fbt* and corresponding *.py* files in the database, to every device. To achieve this we can use the `synchronize.py` script that accompanies the *DINASORE* source code. A configuration file needs to be provided that tells the script where to send the function blocks to, like shown in Listing 5.5. First, `master-fbs-path` dictates where in the server filesystem the function block files that will be sent are located, then an array of `dinasores` with the configurations defined in the *Devices Configuration View*. The `port` used here will be the *SSH* port, the *DINASORE* must have an open *SSH* channel that the server is able to connect to via this port.

```
{
  master-fbs-path: 'sync/fbs',
  dinasores: [
    {
      address: 'x.x.x.x',
      port: 22,
      username: 'root',
      password: 'dinasore',
      dinasore-path: '/root/dinasore'
    },
    {
      address: 'y.y.y.y',
      port: 22,
      username: 'root',
      password: 'dinasore',
      dinasore-path: '/root/dinasore'
    }
  ]
}
```

Listing 5.5: Sync configuration

This file is generated from the configured fields in the *Devices Configuration View*. When the script is ran, it will read the contents of this file in order to connect and send the files to each *DINASORE*.

5.5.2 Send Commands

Once the function block files are in place, the server will send the necessary commands to each *DINASORE* instance to build and start the application. This includes creating the function block instances, writing the initial input values, establishing the connections between event and data ports and finally sending the start command to begin execution. The order in which these messages are issued is important, as connections can only be established between existing nodes. The following two subsections describe how the messages are constructed and what they contain.

5.5.2.1 Build messages

Before the messages can be sent, they need to be built into the format that *DINASORE* is expecting. It consists of a two part message: the first part refers to the configuration name, explained later in Section 5.5.2.2, while the second is the actual message payload containing the wanted command. Each of these parts are preceded by a 3-byte header composed of a `0x50` byte followed by 2 bytes indicating the size of the corresponding content. A representation of this format is presented in Figure 5.7.

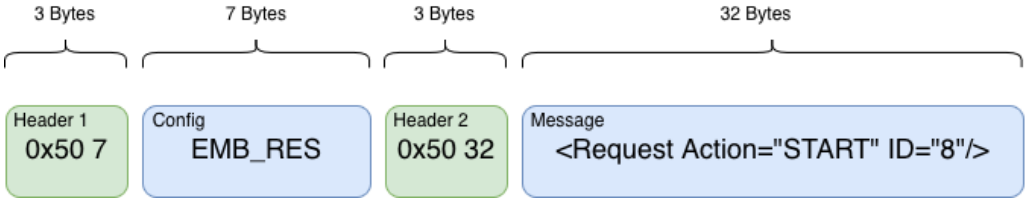


Figure 5.7: *DINASORE* communication format

The *4diac_simulator.py* file, part of the *tests/* folder of *DINASORE*, implements this method in *Python* as shown in Listing 5.6. For this prototype, it was necessary to implement it in *JavaScript*, as shown in Listing 5.7.

```
1  def build_message(message_payload, configuration_name):
2    # build the first part of the header
3    hex_input = '{:04x}'.format(len(configuration_name))
4    second_byte = int(hex_input[0:2], 16)
5    third_byte = int(hex_input[2:4], 16)
6    response_len = struct.pack('BB', second_byte, third_byte)
7    response_header_1 = b''.join([b'\x50', response_len])
8
9    # build the second part of the header
10   hex_input = '{:04x}'.format(len(message_payload))
11   second_byte = int(hex_input[0:2], 16)
12   third_byte = int(hex_input[2:4], 16)
13   response_len = struct.pack('BB', second_byte, third_byte)
14   response_header_2 = b''.join([b'\x50', response_len])
15
16   # join the 3 parts
17   response = b''.join([response_header_1, configuration_name, response_header_2,
         message_payload])
```

```
  return response
```

Listing 5.6: build_message() in Python

```
function build_message(message_payload, configuration_name) {
  const configBuffer = Buffer.from(configuration_name, 'utf-8');
  const messageBuffer = Buffer.from(message_payload, 'utf-8');

  // Header 1: [0x50][len_hi][len_lo]
  const header1 = Buffer.alloc(3);
  header1[0] = 0x50;
  header1.writeUInt16BE(configBuffer.length, 1);

  // Header 2: [0x50][len_hi][len_lo]
  const header2 = Buffer.alloc(3);
  header2[0] = 0x50;
  header2.writeUInt16BE(messageBuffer.length, 1);

  return Buffer.concat([
    header1,
    configBuffer,
    header2,
    messageBuffer
  ]);
}
```

Listing 5.7: build_message() in JavaScript

5.5.2.2 Send Messages

Now that we have defined how to build the messages, we can explore the sequence of commands that need to be sent to each *DINASORE* instance and what configuration name should be used in each. The server connects to each instance and transmits these messages sequentially. The order of message transmission is critical, as connections can only be established between existing nodes. The configuration name used in each message will be determined by whether the command is bound to any resource. In *DINASORE* the existing resource is *EMB_RES* as mentioned before.

First we need to send the 2 commands in Listing 5.8 to every device. A `QUERY` request and a `CREATE` request that will create a *EMB_RES* resource which will be automatically connected to the *INIT* port of all other function block nodes to initialize them. Both these commands will have an empty configuration name, as *EMB_RES* has not been created yet and therefore cannot be bounded to.

```
<Request Action="QUERY" ID="0">
  <FB Name="*" Type="*"/>
</Request>

```

```
<Request Action="CREATE" ID="1">
  <FB Name="EMB_RES" Type="EMB_RES"/>
</Request>
```

Listing 5.8: Setup deployment commands

Next, depending on which commands are mapped to each machine, we `CREATE` all the mapped nodes and `WRITE` the initial input values with the commands in Listing 5.9. The configuration name in these and all following commands will be "*EMB_RES*" as they need to be bound to it in order to be initialised and ran.

```
<Request Action="CREATE" ID="2">
  <FB Name="SENSOR_SIMULATOR" Type="SENSOR_SIMULATOR"/>
</Request>

<Request Action="WRITE" ID="3">
  <Connection Destination="SENSOR_SIMULATOR.OFFSET" Source="12"/>
</Request>

<Request Action="CREATE" ID="4">
  <FB Name="MOVING_AVERAGE" Type="MOVING_AVERAGE"/>
</Request>

<Request Action="WRITE" ID="5">
  <Connection Destination="MOVING_AVERAGE.WINDOW" Source="5"/>
</Request>
```

Listing 5.9: `CREATE` nodes deployment commands

Then the connections are created between the mapped nodes. A `Source` a `Destination` must be provided, with the format `"NODE_NAME.SLOT_NAME"`, using the commands in Listing 5.10.

```
<Request Action="CREATE" ID="6">
  <Connection Destination="SENSOR_SIMULATOR.READ_0" Source="MOVING_AVERAGE.RUN"/>
</Request>

<Request Action="CREATE" ID="7">
  <Connection Destination="SENSOR_SIMULATOR.VALUE" Source="MOVING_AVERAGE.VALUE"/>
</Request>
```

Listing 5.10: `CREATE` links deployment commands

Finally the `START` command in Listing 5.11 can be issued, which will request for the start of the program execution. For this command the configuration name used will also be "*EMB_RES*" as it is this resource that will begin the execution of the program.

```
1 <Request Action="START" ID="8"/>
```

Listing 5.11: `START` deployment command

After each request is sent, a response will be returned by the *DINASORE*. It is important for the IDE to listen for these responses and only send the next request once the response corresponding to the previous request is received. The responses have the format shown in Listing 5.12. The `ID` of the response will be the same of the associated request. The format of the responses is the same as the requests and the configuration name that is used is empty.

```
1 <Response ID="8"/>
```

Listing 5.12: `Response` after a deployment command is sent

5.5.3 Console

To give the team visibility into the deployment process right from the IDE, without having to log into the server, a console is provided in the *Console View* that displays a live log of each action taken during deployment. There are 2 main things being logged:

- **Synchronise**: The output of running the *Python* script is displayed so that the team see that the *SSH* communication is being done properly.
- **Send Commands**: As commands are sent and responses are received, they are displayed, allowing the team to make sure each command is formulated correctly, being processed and a response is issued.

5.5.4 Watch

Once the application is running, the values of the node outputs are continuously retrieved from the *DINASORE* instances and displayed in the *Graph View*, providing immediate feedback on the state of the running application. This is particularly useful for debugging and validating the behaviour of the application during development. In order to get these values, the IDE must tell *DINASORE* instances which output slots it wants to watch and then request the values. In this prototype, for simplicity, all data outputs are watched and we need to send a watch creation request for each. Note in Listing 5.13 that the `ID` is reset after deploying.

```
1 <Request ID="0" Action="CREATE">
2   <Watch Source="SENSOR_SIMULATOR.VALUE" Destination="*" />
3 </Request>
4
5 <Request ID="1" Action="CREATE">
```

```
6   <Watch Source="MOVING_AVERAGE.VALUE_MA" Destination="*" />
7 </Request>
```

Listing 5.13: `CREATE` watches after deploying

After creating the watches we can send a request to receive the values using the command in Listing 5.14. This will return the response in Listing 5.15, which by parsing the values, allows the prototype to display them in the *Graph View*, on the right side of the corresponding output slots, demonstrated in Figure 5.8.

```
1 <Request ID="2" Action="READ">
2   <Watches/>
3 </Request>
```

Listing 5.14: Request to `READ` watches

```
1  <Response ID="2">
2    <Watches>
3      <Resource name="EMB_RES">
4        <FB name="SENSOR_SIMULATOR">
5          <Port name="VALUE">
6            <Data value="13.75" />
7          </Port>
8        </FB>
9        <FB name="MOVING_AVERAGE">
10         <Port name="VALUE">
11           <Data value="13.3" />
12         </Port>
13       </FB>
14     </Resource>
15   </Watches>
16 </Response>
```

Listing 5.15: Response to `READ` watches

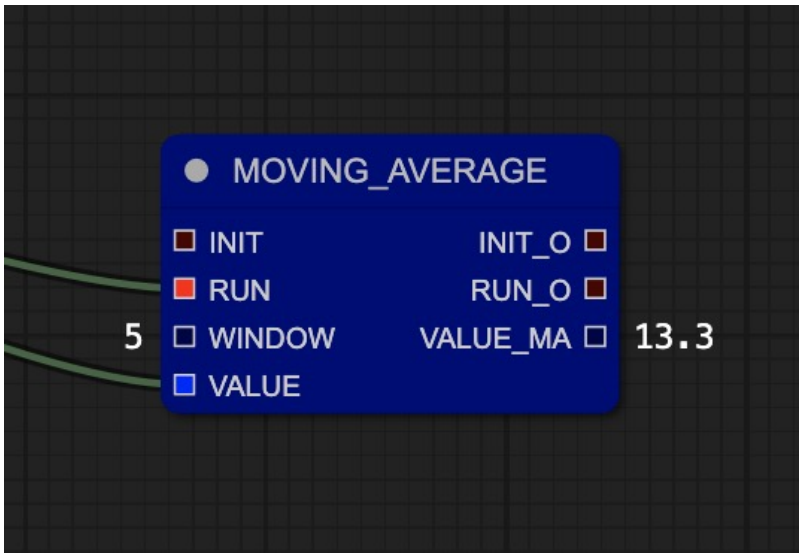


Figure 5.8: Watching an output slot in the *Graph View*

5.6 Distribution

Chapter 6

Validation

In order to validate that the proposed solution achieves the proposed objectives, effectively addressing the gap previously identified, the prototype was tested by approaching it from two complementary perspectives: objective testing, which seeks to find measurable and quantifiable aspects of the system's behaviour and design decisions, and subjective testing, which asks real users to interact with the prototype, giving feedback on their perception and experience with the system.

6.1 4diac IDE User Experience

Before beginning testing of the proposed solution, a study on the user experience of working with *4diac IDE* as the underlying development environment for *IEC 61499* was conducted. The participants of the questionnaire were students from the Master in Electrical and Computer Engineering (MEEC) at Faculdade de Engenharia da Universidade to Porto (FEUP), enrolled in the course of Information Systems (SINF), in which *4diac IDE* was used as the primary tool for modelling FBD programs, using *DINASORE* as the RTE.

6.1.1 Questionnaire Design

Firstly the questionnaire asked the participants if the SINF course was the first time they have used *4diac IDE*. This was important to differentiate answers based on prior experience.

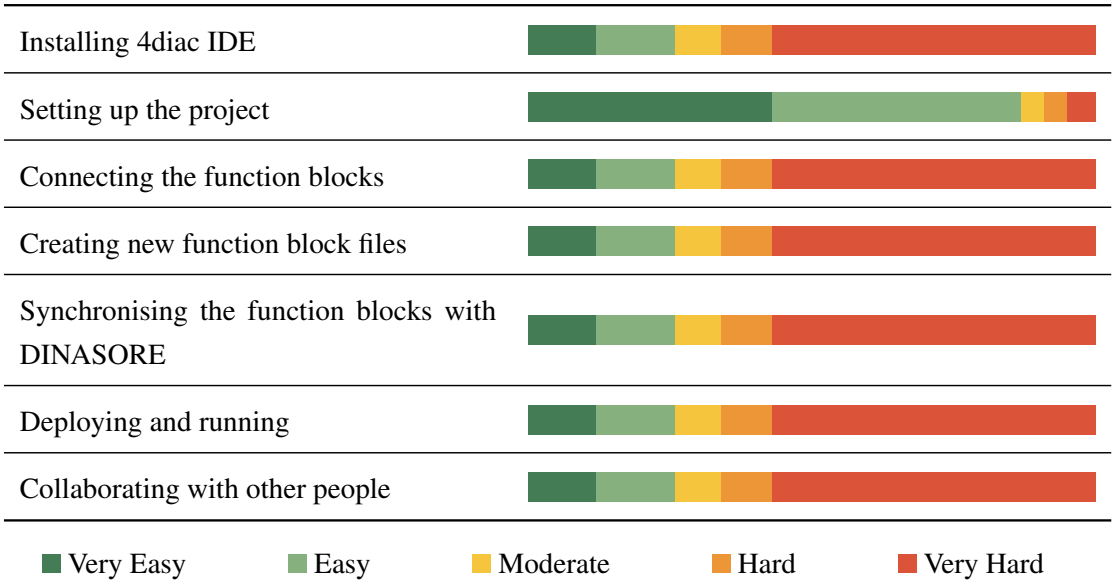
Then followed a Likert [8] table on the difficulty of doing several different tasks using this IDE. Ranging from *Very Easy* to *Very Hard*, this section had the intent of understanding how easy and straightforward each step of *IEC 61499* development was. If a specific step was difficult in *4diac IDE*, it will be interesting to see how it compares with *PTERO*.

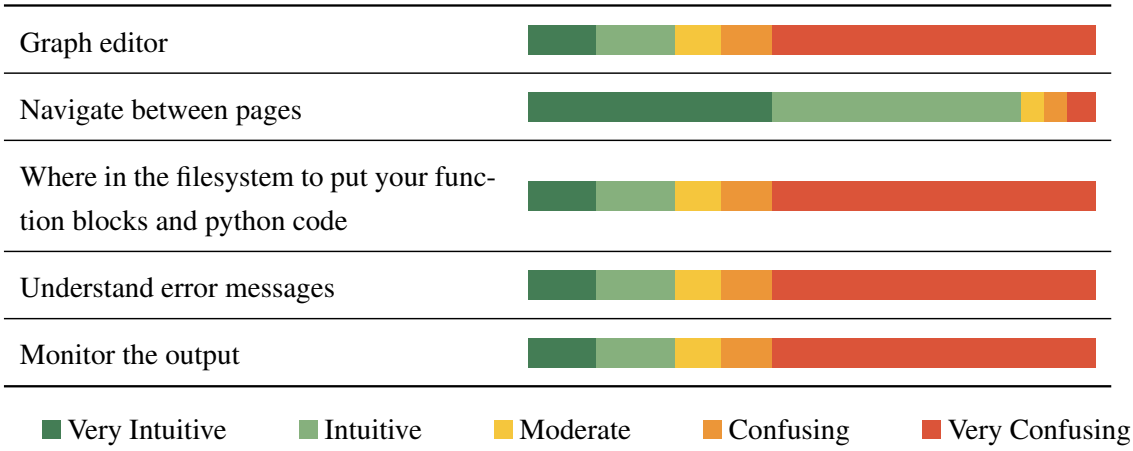
Next, another Likert table, this time on how intuitive it was to understand each part of the IDE. This way the design decisions of *4diac IDE* can be critically analysed.

The questionnaire concluded with the following 3 open text questions, asking participants to provide feedback in their own words on the aspects they considered most relevant in their experience with the tool.

- What did you **like** most about using 4diac IDE?
- What were the **biggest challenges** you faced?
- Any additional comments?

6.1.2 Results





6.1.3 Discussion

6.2 Test Program Design

The following FBD program design was used in both the objective and subjective validation tests.

6.2.1 Available resources

These are the resources available from the start to the tested IDE.

- 1 *DINASORE* instance. More could be used but in order to make user testing simpler, a single instance was deemed appropriate.
- 3 function blocks, defined by 2 files each. The files are taken from the official *DINASORE*'s tutorial *'Sensorization "Hello World!"'*. They will need to be imported into the IDE and the *DINASORE* instance before the program can be deployed
  - SENSOR_SIMULATOR.fbt and SENSOR_SIMULATOR.py
  - MOVING_AVERAGE.fbt and MOVING_AVERAGE.py
  - CONCATENATE_REALS.fbt and CONCATENATE_REALS.py

6.2.2 Event and Data Flow

The following flow, as represented in *4diac IDE*, should be defined in the IDE and then sent to the RTE:

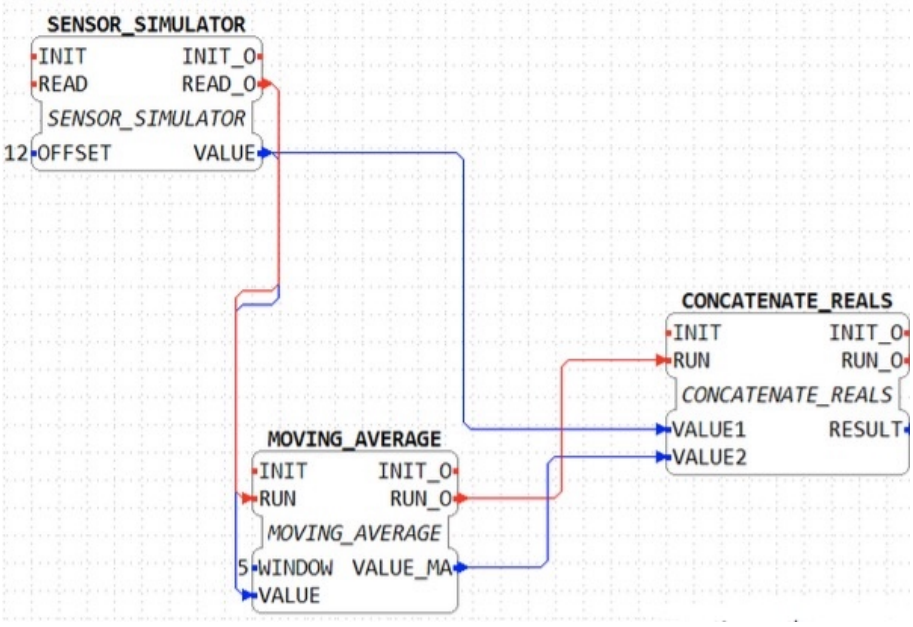


Figure 6.1: Test design represented in *4diac IDE*

6.2.3 Expected Output

After deploying, the IDE should display the generated output values of the output data slots.

6.3 Objective Validation

Objective validation focuses on measurable criteria that can be validated independently of user opinion. These tests were designed to verify if the prototype, and by extension the proposed solution, does not fail to meet the functional requirements of a *IEC 61499* IDE and is able to confidently achieve the goals that it sets out for. Besides being able to function, these tests also measure performance metrics that can be directly compared with *4diac IDE*.

6.3.1 Functional Correctness

The prototype was subjected to functional tests to confirm that all core features behave as specified. It was tested by implementing and deploying the test program defined in Section 6.2. As a control, the same was performed in *4diac IDE*. The steps taken are as follows:

1. **Configured available *DINASORE***
   - **4diac IDE**: In the *System Configuration* tab, added a device and set its IP address and the port where *DINASORE* is running, then connected it to the *Ethernet* tunnel.
   - **PTERO**: In the *Config* tab, added a device and set its IP address and the port where *DINASORE* is running, as well as the SSH details of the device which will be used to sync the function blocks files.
2. **Imported the function block files**
   - **4diac IDE**: Copied and pasted the given files to `4diac-ide/typelibrary/(new_folder)`.
   - **PTERO**: Copied and pasted the given files in the *Function Blocks* tab.
3. **Designed the flow**
   - **4diac IDE**: In the *App View* tab, dragged and dropped the wanted function blocks, then connected the event and data slots. Double-clicked input slots to assign input value.
   - **PTERO**: In the *Graph* tab, right-clicked and selected the wanted function blocks to add them, then connected the event and data slots. Right-clicked nodes to assign input value.
4. **Mapped to *DINASORE***
   - **4diac IDE**: Right-clicked nodes to map them to the device. They became color coded to show which device they were mapped to.

- **PTERO**: Right-clicked nodes to map them to the device. They became color coded to show which device they were mapped to.

5. **Synced files with *DINASORE***
   - **4diac IDE**: Called the `synchronize.py` manually, feeding it the needed SSH details.
   - **PTERO**: This was done automatically and without any needed setup when deploying.
6. **Deployed**
   - **4diac IDE**: Pressed the *Deploy* button. The sent commands and associated responses were displayed in a console.
   - **PTERO**: Pressed the *Deploy* button. The sent commands and associated responses were displayed in a console.
7. **Monitored the output**
   - **4diac IDE**: The *Monitor values* option was enabled, and the values were displayed on the right side of the output data slots.
   - **PTERO**: The values were displayed on the right side of the output data slots.

Besides verifying that the output values are being monitored correctly, following the *Deployed* step, the commands displayed in the console were directly compared between IDEs. It was verified that both were sending exactly the same commands, the headers as well as the contents were exactly the same, and the responses were no different. These results confirmed that the prototype correctly handles the development of *IEC 61499* programs.

6.3.2 Performance

This solution not only aims to be able to do the same workload as *4diac IDE*, but also have capabilities and an inherent design that justify its existence. Therefore, this section compares the performance of several real world tasks between the IDEs. The listed tasks are the ones that demonstrated to be significantly different between solutions, others were analysed but proved to not be worth the discussion.

- **Install**
  - **4diac IDE**: Install a desktop app. There is no compatible version for macOS.
  - **PTERO**: Install a *Node.js* server. By working in a web browser, it is crossplatform without much development effort, which allows macOS users to use it. The downside is that it is harder to install for just local development.
- **Open**

  - **4diac IDE**: Average of 4 seconds to boot the application in a mid level computer. As it is written in *Java*, the boot time cannot be improved much.
  - **PTERO**: Average of less than a second to load webpage. The use of lightweight libraries and the move of heavy workloads to the server allow the page to load in an instant.

- **Create and edit function block files**
  - **4diac IDE**: While it does have a way to create and edit the *.fbt* files inside the editor, the code editor is not targeted against *DINASORE* and therefore does not support *.py* files.
  - **PTERO**: Has a text editor for both types of files.

- **Collaboration**
  - **4diac IDE**: An external periodic version control system like *Git* needs to be used. Structured data in formats like *XML* is not handled properly by these types of version control system, which results in frequent merge conflicts [2]. The following test was conducted:
    1. A project has a graph with 1 function block added. 2 users are working on the graph.
    2. User #1 deletes the function block.
    3. User #2 modifies the position of the function block.
    4. User #1 commits and pushes.
    5. User #2 tries to pull the changes but gets a merge conflict
  - **PTERO**: Has a realtime collaboration system. Any change made is immediately propagated to every client, and resolved by a CRDT, ensuring no conflicts need to be handled manually and damage is minimised.

- **Synchronising files with the *DINASOREs***
  - **4diac IDE**: Required to manually use an external script before deployment.
  - **PTERO**: The server itself will connect and synchronise the files with the *DINASOREs*. This happens automatically when attempting to deploy, ensuring that deployments never fail due to the human error of forgetting to sync the files.

- **Remote deployment**
  - **4diac IDE**: Every IDE must have a valid connection with each available *DINASORE*, as well as *SSH* access. This gets increasingly complicated if the user want to make a deployment while in a different network. A way to expose each individual device needs to be design.

- **PTERO**: By having the server as the middleware, only it must be exposed for the users to connect. With only the server permanently having direct connection with the *DINASORE* devices, no individual handling is needed.

6.4 Subjective Validation

Chapter 7

Discussion

References

[1] *4diac IDE Website*. URL: https://eclipse.dev/4diac/4diac_ide/.

[2] E. Axelsson and Sérgio Batista. "Version Control of structured data: A study on different approaches in XML". In: 2015. URL: https://www.semanticscholar.org/paper/Version-Control-of-structured-data%3A-A-study-on-in-Axelsson-Batista/323392e2fee60020d73f949fc996c31bf8c5b7ee (visited on 05/31/2026).

[3] Young-Jo Cho et al. "Function Block Diagram Approach for Manufacturing Process Control Programming". en. In: *IFAC Proceedings Volumes* 26.2 (July 1993), pp. 461–464. ISSN: 14746670. DOI: 10.1016/S1474-6670(17)48510-7. URL: https://linkinghub.elsevier.com/retrieve/pii/S1474667017485107 (visited on 01/28/2026).

[4] Marcelo V. Garcia et al. "Engineering tool to develop CPPS based on IEC-61499 and OPC UA for oil&gas process". en. In: *2017 IEEE 13th International Workshop on Factory Communication Systems (WFCS)*. Trondheim, Norway: IEEE, May 2017, pp. 1–9. ISBN: 978-1-5090-5788-7. DOI: 10.1109/WFCS.2017.7991969. URL: http://ieeexplore.ieee.org/document/7991969/ (visited on 01/30/2026).

[5] *IEC 61131-3*. en. Page Version ID: 1339669093. Feb. 2026. URL: https://en.wikipedia.org/w/index.php?title=IEC_61131-3&oldid=1339669093 (visited on 05/11/2026).

[6] *IEC 61499 Wikipedia*. URL: https://en.wikipedia.org/wiki/IEC_61499.

[7] Steven Kelly. "Collaborative Modelling with Version Control". In: ed. by Martina Seidl and Steffen Zschaler. Vol. 10748. Book Title: Software Technologies: Applications and Foundations Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2018, pp. 20–29. ISBN: 978-3-319-74729-3 978-3-319-74730-9. DOI: 10.1007/978-3-319-74730-9_3. URL: http://link.springer.com/10.1007/978-3-319-74730-9_3 (visited on 06/04/2026).

[8] R. Likert. "A TECHNIQUE FOR THE MEASUREMENT OF ATTITUDE SCALES". In: 1932. URL: https://www.semanticscholar.org/paper/A-TECHNIQUE-FOR-THE-MEASUREMENT-OF-ATTITUDE-SCALES-Likert/c7c4b379b2ca0d0c134bc3c0f114a326e (visited on 06/01/2026).

[9] Tuojian Lyu et al. "Towards Web Platform for Cloud-Based Virtual Commissioning of IEC 61499 Distributed Automation Systems". en. In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. Padova, Italy: IEEE, Sept. 2024, pp. 1–4. ISBN: 979-8-3503-6123-0. DOI: 10.1109/ETFA61755.2024.10710351. URL: https://ieeexplore.ieee.org/document/10710351/ (visited on 12/17/2025).

[10] László Monostori. "Cyber-physical Production Systems: Roots, Expectations and R&D Challenges". en. In: *Procedia CIRP* 17 (2014), pp. 9–13. ISSN: 22128271. DOI: 10.1016/j.procir.2014.03.115. URL: https://linkinghub.elsevier.com/retrieve/pii/S2212827114003497 (visited on 01/28/2026).

[11] Linna Pang et al. "Formal verification of function blocks applied to IEC 61131-3". en. In: *Science of Computer Programming* 113 (Dec. 2015), pp. 149–190. ISSN: 01676423. DOI: 10.1016/j.scico.2015.10.005. URL: https://linkinghub.elsevier.com/retrieve/pii/S0167642315002981 (visited on 01/29/2026).

[12] E. Pereira, João C. P. Reis, and G. Gonçalves. "DINASORE: A Dynamic Intelligent Reconfiguration Tool for Cyber-Physical Production Systems". In: 2020. URL: https://www.semanticscholar.org/paper/DINASORE%3A-A-Dynamic-Intelligent-Reconfiguration-for-Pereira-Reis/8337325c710a6f08eb9fc85855852bc8aaa2e9d1 (visited on 06/02/2026).

[13] Gonçalo Pinto et al. "Digital Factory: An Industrial Case Study for Function Block-Oriented Development". en. In: *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. Aveiro, Portugal: IEEE, Oct. 2023, pp. 01–06. ISBN: 979-8-3503-1161-7. DOI: 10.1109/WF-IoT58464.2023.10539486. URL: https://ieeexplore.ieee.org/document/10539486/ (visited on 01/29/2026).

[14] Oana Rohat and Dan Popescu. "Remote web-based execution of IEC 61499 function blocks". en. In: *Proceedings of the 2014 6th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*. Bucharest, Romania: IEEE, Oct. 2014, pp. 33–38. ISBN: 978-1-4799-5478-0 978-1-4799-5479-7. DOI: 10.1109/ECAI.2014.7090220. URL: http://ieeexplore.ieee.org/document/7090220/ (visited on 01/28/2026).

[15] Oliver Schmid, Agnes Lisowska Masson, and Béat Hirsbrunner. "Real-time collaboration through web applications: an introduction to the Toolkit for Web-based Interactive Collaborative Environments (TWICE)". en. In: *Personal and Ubiquitous Computing* 18.5 (June 2014), pp. 1201–1211. ISSN: 1617-4909, 1617-4917. DOI: 10.1007/s00779-013-0729-0. URL: http://link.springer.com/10.1007/s00779-013-0729-0 (visited on 06/04/2026).

[16] *ScienceDirect*. URL: https://www.sciencedirect.com.

[17] Radimir Sorokin, Sandeep Patil, and Valeriy Vyatkin. "Novel development tool for IEC 61499 based on domain-specific languages". en. In: *IFAC-PapersOnLine* 55.2 (2022), pp. 439–444. ISSN: 24058963. DOI: 10.1016/j.ifacol.2022.04.233. URL: https://linkinghub.elsevier.com/retrieve/pii/S2405896322002348 (visited on 01/28/2026).

[18] Tomás Torres, Gil Gonçalves, and Rui Pinto. "Literature Review on Cloud-Based Service-Oriented Architecture for IEC 61499 Distributed Control Systems:" en. In: *Proceedings of the 15th International Conference on Cloud Computing and Services Science*. Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 174–181. ISBN: 978-989-758-747-4. DOI: 10.5220/0013293400003950. URL: https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0013293400003950 (visited on 01/28/2026).

[19] *VSCode Web*. URL: https://code.visualstudio.com/docs/setup/vscode-web.

[20] Thomas Weber, Alois Zoitl, and Heinrich HuBmann. “Usability of Development Tools: A CASE-Study”. en. In: *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. Munich, Germany: IEEE, Sept. 2019, pp. 228–235. ISBN: 978-1-7281-5125-0. DOI: 10.1109/MODELS-C.2019.00037. URL: https://ieeexplore.ieee.org/document/8904489/ (visited on 01/30/2026).

[21] *Zed Roadmap*. URL: https://zed.dev/roadmap.

[22] Alois Zoitl, Thomas Strasser, and Gerhard Ebenhofer. “Developing modular reusable IEC 61499 control applications with 4DIAC”. en. In: *2013 11th IEEE International Conference on Industrial Informatics (INDIN)*. Bochum, Germany: IEEE, July 2013, pp. 358–363. ISBN: 978-1-4799-0752-6. DOI: 10.1109/INDIN.2013.6622910. URL: http://ieeexplore.ieee.org/document/6622910/ (visited on 01/29/2026).